Sjávs

John Askham

ForritanarVerkførði

Course ID: 5020.24

Lesandi B.Sc. í KT-verkføði

Fróðskaparsetur Føroya

20-12-2024

FRÓÐSKAPARSETUR
FØROYA

Contents

1 Abstract **1**

2 Introduction **2**

3 Communication **3**

3.1 Understanding the Customer's Needs and Requirements 3

3.2 Requirement Gathering and Analysis 3

3.3 Communication Plan and Strategy 4

4 Planning **5**

4.1 Project Plan and Timeline 5

4.2 Methodology Selection 6

4.3 Requirement prioritization 6

4.4 Resource allocation 6

5 Modelling **7**

5.1 System architecture 7

5.2 Data Modelling 8

5.2.1 Use case modelling 10

5.3 Activity diagrams 11

6 Construction **13**

6.1 Development 13

6.1.1 Game Logic Sequence 13

6.2 Testing and debugging 17

6.3 Integration and verification 18

6.3.1 Sjavs Integration 18

6.3.2 Clerk 18

7 Deployment **21**
7.1 Deployment Strategy 21
7.2 Challenges and Troubleshooting 22
7.3 Release management 23
7.4 Describe the final product 23

8 Feedback **24**
8.1 User Testing and Feedback 24
8.1.1 Customer Feedback 24
8.2 Missing Features and Future Implementation Plan 25
8.2.1 Tournament Functionality 25
8.2.2 Multiplayer Capabilities 25
8.2.3 Post Stats on Social Media 25
8.2.4 Game Tutorial 26
8.2.5 Improved Interface 26

9 Review **27**
9.1 Describe the Overall Team Experience 27
9.1.1 What Went Wrong 27
9.1.2 What Worked Well 27
9.1.3 How Could You Improve 28
9.1.4 Final Verdict of the Team 28

10 Conclusion **29**
10.1 Summary of Key Findings and Recommendations 29
10.2 Lessons Learned 29
10.3 Future Work 29

11 Appendix A **30**

12 Appendix B **31**

12.1 First Meeting 31

12.2 Second Meeting 32

12.3 Third Meeting 32

1 Abstract

This report summarizes the development of a web-based application for the traditional card game Sjavs, aimed at modernizing gameplay for accessibility through web browsers. The project focused on creating a functional prototype with core game features, laying the groundwork for future enhancements such as multiplayer support and tournament modes.

The application was developed using Next.js with TypeScript for functional programming, Clerk for user authentication, and Supabase for database operations, enabling features like player statistics tracking and game lobbies.

While a single-player version of the game was successfully implemented, the project faced significant challenges, including time constraints and technical hurdles, which prevented the realization of multiplayer functionality.

Despite these limitations, the project offers valuable insights into the complexities of learning new tools and frameworks, emphasizing the importance of early deployment, thorough testing, and resource management. This work serves as a solid foundation for future development to achieve the vision of a competitive, interactive digital Sjavs platform.

2 Introduction

In this project, we were tasked with creating a digital version of the traditional card game Sjavs, designed to be accessible via popular web browsers such as Google Chrome, Mozilla Firefox, and Safari. Over an eight-week timeline, our goal was to develop a prototype that supports up to four players in a single game, maintaining the core mechanics of Sjavs while leveraging modern web technologies.

This project aimed to modernize a classic card game by improving its accessibility and engagement, providing a platform for both casual and competitive play. This report details the development process, discusses the technical challenges and solutions encountered, and showcases the final product delivered within the project's constraints.

3 Communication

We maintained consistent communication with our customer, Pál Vitalis Joensen, who envisioned a web application for the Sjavs card game. Initially, Pál provided us with a written report detailing his vision, but we quickly recognized the need for a meeting to clarify key requirements and establish a manageable scope for the project.

3.1 Understanding the Customer's Needs and Requirements

During our first meeting with Pál, we clarified his expectations for the project and identified the core functionalities required for the initial prototype. The main focus was to deliver a playable version of Sjavs that could serve as a foundation for future development.

Key features discussed included:

- **Multiplayer Functionality:** The ability for multiple players to participate in a game.
- **Tournament Mode:** A feature to host tournaments with entry fees, allowing players to compete for a prize pool. A portion of the proceeds would go toward charitable causes.
- **Player Statistics:** The inclusion of a profile system to track individual player stats, such as wins, losses, and games played.

3.2 Requirement Gathering and Analysis

The project requirements were refined during subsequent meetings with Pál. The key takeaways included:

- A functional multiplayer card game with adherence to Sjavs rules.
- A scalable system capable of supporting tournaments and competitive play in the future.
- The ability for players to view and track their performance through a statistics dashboard.

These requirements formed the backbone of the project and guided our development efforts, with additional features prioritized for future updates.

3.3 Communication Plan and Strategy

Our initial plan was to hold frequent meetings with Pál and provide regular updates via emails to keep him informed about the progress of the project. These updates were intended to ensure alignment with his expectations and maintain clear communication throughout the development process.

We ended up having three meetings with Pál, and a few e-mail exchanges between meet up's and the progress of the application.

4 Planning

After meeting with Pál, we gained a clearer understanding of the project requirements, enabling us to effectively plan our approach. We decided to adopt an agile work methodology, selecting a framework suited to the project's needs. Additionally, we established tools and processes to track progress and ensure the project would be delivered on time to meet the customer's expectations.

We had a few selection's of agiles's method's, which we could use for the project. We were not going to use Extreme Programming (XP) because of our group size. We then had the selection between Scrum or Kanban, each one has it's own method of working with sprints and keeping projects on track.

We decided to use Scrum because it sounded the best for us, using the standup meetings and the use of the Scrum Master and the Project Manager, we should have been able to keep a high focus on each part of the project, helping us to keep the timeline of the project well.

For project management, we decided to use Trello.com, an interactive board that allowed us to customize it extensively without requiring a subscription. Trello also supports multiple free members within the same workspace, making it a cost-effective solution for our small team. This tool enabled us to efficiently plan and organize our user stories and sprints, ensuring smooth collaboration and progress tracking throughout the project.

4.1 Project Plan and Timeline

For the project, we chose Next.js as our framework, utilizing TypeScript and JavaScript to develop the Sjavs application. This was a bold decision, as it ventured into unfamiliar territory—our experience with JavaScript was limited, and we had never worked with a framework before. Despite this, we decided to take the risk, aiming to program the application in a functional style, deliberately avoiding Object-Oriented Programming (OOP).

In our project, we are using Next.js to build a Single Page Application (SPA). While Next.js supports Multi-Page Applications (MPAs), we are utilizing its client-side navigation features. This means that instead of generating new HTML files for each route, we navigate between page.tsx files in the pages/ directory, dynamically loading and rendering data for the user on the same single-page context. This approach gives us the benefits of faster transitions and a more seamless user experience while still leveraging Next.js's features for data fetching and optimization.

For version control, we initially used Bitbucket as our repository. However, during the deployment phase, we switched to GitHub, as it offered better support for our deployment requirements.

To manage the project's data, we chose to use Supabase as our online database management system. This decision allowed our team to stay synchronized and work with the same data seamlessly across all phases of the project.

As students managing two classes, we implemented a strategy to alternate our focus between this project and another course. We allocated one week to work on this project and the following week to the other, ensuring balanced progress on both. Given the eight-week duration of this course, we adapted the Scrum methodology by creating four short sprints with cooldown periods in between to reflect our alternating schedule.

4.2 Methodology Selection

We chose to adopt the Scrum agile methodology for this project. Despite being a small team of three, we found Scrum to be an effective way to stay focused and up-to-date on our tasks. Roles were distributed as follows: I took on the role of Project Manager, Hanus served as the Scrum Master, and Magnus worked as the primary developer.

By utilizing daily meetings, a storyboard, and sprints, we aimed to maintain clear communication and adhere to the project timeline effectively.

4.3 Requirement prioritization

The first priority of the project is to get a prototype for the customer, where you are able to play a game of Sjavs.

This would then lead into building the multiplayer logic for the game, allowing four people to play together.

4.4 Resource allocation

We all set our own focus at the start of the project. I myself would handle the frontend of the project, Magnus would create the backend of the game logic, and Hanus would create the login and DBMS for the server. When Magnus was done with the backend logic, he went to help with the DBMS to attempt to make multiplayer support for the game.

5 Modelling

We intend to have a simple layout for the project to ensure that it does not confuse the users of the application. Keeping things minimal will also improve the user experience. This section highlights the layout of the system and how it works.

5.1 System architecture

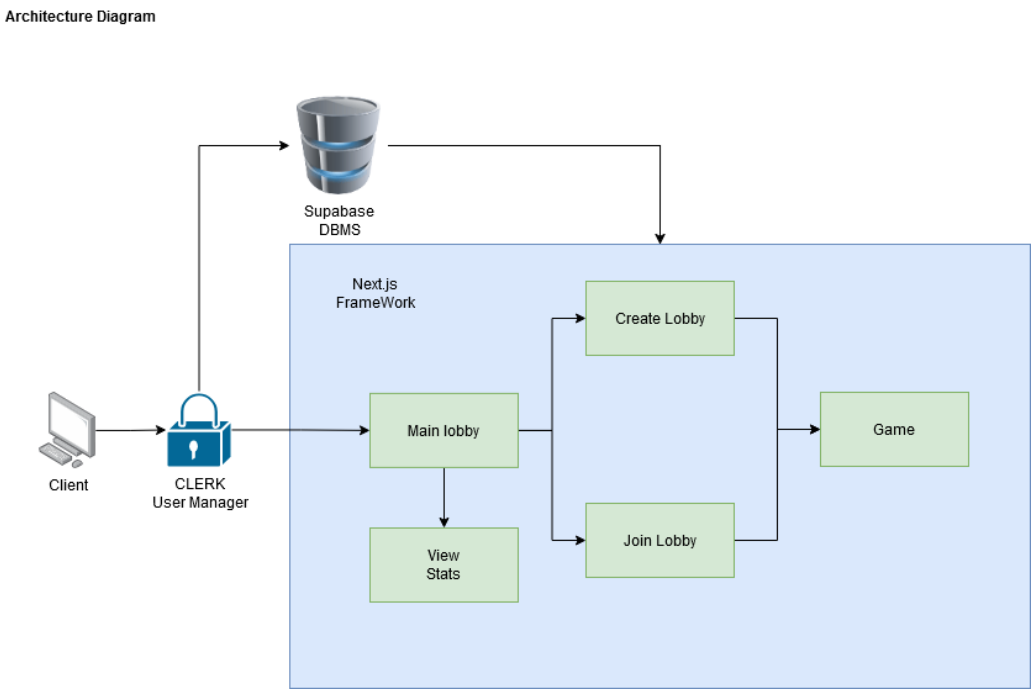

Figure 1: Architecture Diagram

The **Architecture Diagram** provides a high-level overview of the system's structure. It illustrates the key components of our application and shows the navigation flow available to the user, such as accessing the Main Lobby, creating or joining a game lobby, and viewing stats. Additionally, it highlights the key services like **Clerk** for user management and **Supabase** for database management.

This simplified diagram focuses on how the application's main components connect and navigate from one point to another without delving into detailed backend interactions or user-specific actions.

5.2 Data Modelling

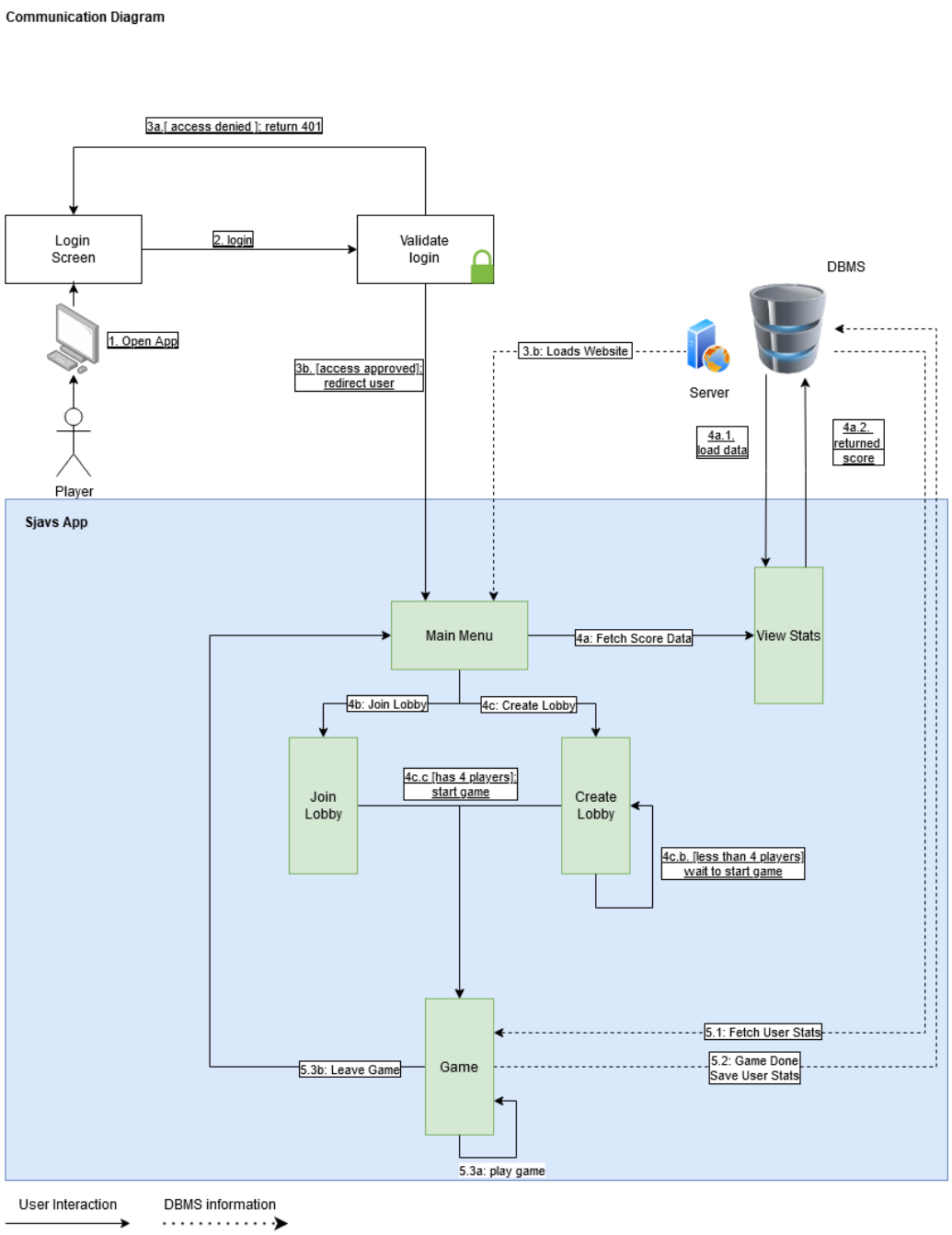

Figure 2: Communication Diagram

This online application supports multiplayer functionality, requiring careful handling of user information to ensure seamless access and interaction. Using **Clerk** for user management, we securely store user data in our **Supabase** database.

When a user wishes to join a lobby, they are provided with a unique code (e.g., 1234) by a friend, which links them to the correct lobby. The Database Management System (**DBMS**) registers the user and maintains records of the players in each lobby. Once the DBMS detects that four players have joined a specific lobby, it enables the host to start the game, ensuring all prerequisites for gameplay are satisfied.

Upon completion of a game, each user's data is updated and saved back into the **DBMS**. This enables players to view their personal statistics, such as the total number of games played, wins, and losses.

Communication Diagram Overview: The communication diagram (Figure 2) provides a detailed visualization of the interaction between the user, the system, and the database. It captures the following key processes:

- **Login Process:** Users authenticate through the login screen, and their credentials are validated using Clerk. Successful authentication redirects the user to the main menu.
- **Main Menu Navigation:** From the main menu, users can create or join a lobby, view their statistics, or start a game. These actions trigger relevant system responses and database queries.
- **Lobby Management:** The DBMS tracks players joining or leaving lobbies, ensuring that game initialization occurs only when all necessary conditions (e.g., four players) are met.
- **Gameplay and Statistics Management:** During gameplay, the system dynamically updates user actions and progress in the database. After the game, statistics are saved and made available for users to review.

This communication flow ensures efficient interaction between the application and the database, providing a smooth user experience while maintaining data integrity and synchronization.

5.2.1 Use case modelling

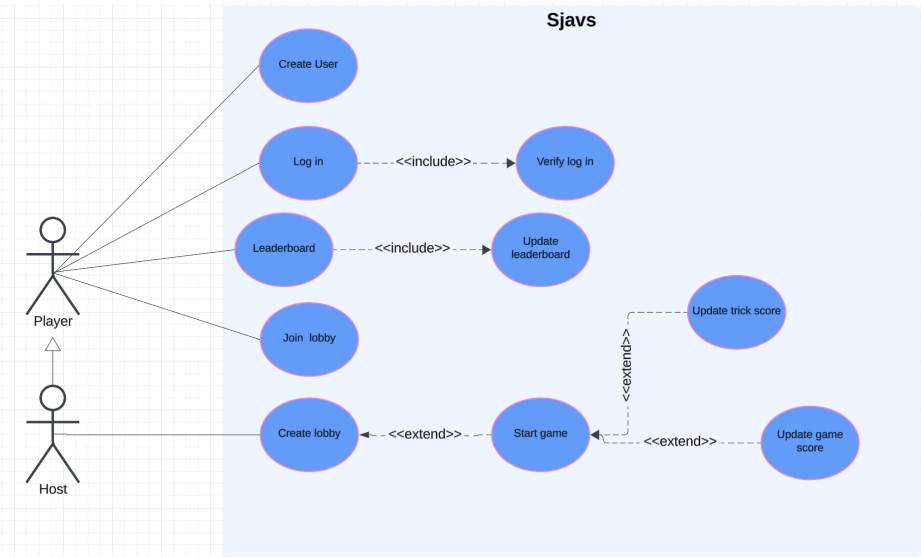

Figure 3: Use Case Diagram

In our application, the user (player) is able to navigate throughout all the menus in the game. The user can interact with all elements on the frontend of the application and can host their own games or lobbies.

This functionality requires that the user has an account and is logged into the system to perform these actions. For example:

- The user can log in, view leaderboards, and join game lobbies.
- The host can create game lobbies and start games once all conditions are met.
- The system ensures all user interactions (e.g., updating scores and leaderboards) are handled correctly.

The use case diagram (Figure 3) illustrates these functionalities and the interactions between the system and the users.

5.3 Activity diagrams

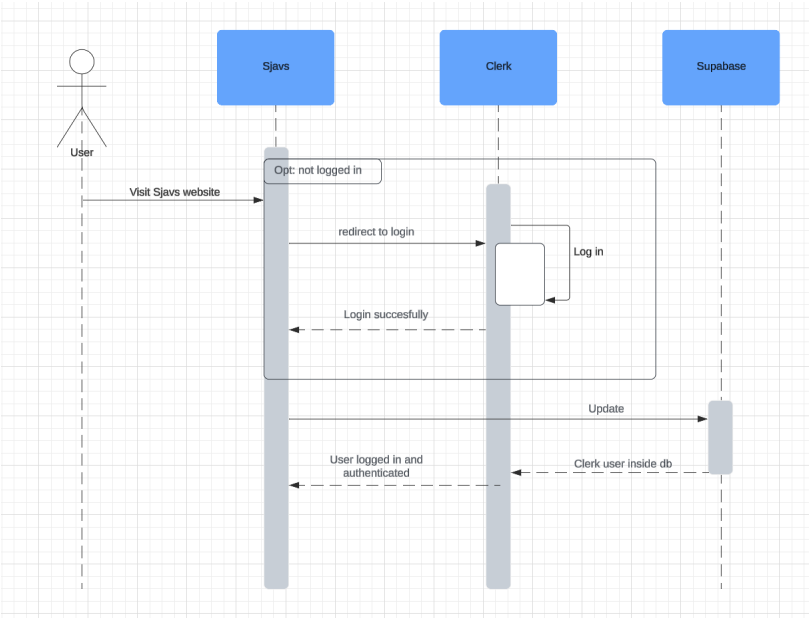

Figure 4: Login sequence diagram

The login system of the project was handled with Clerk, which allowed for user safety because our own DBMS does not store the user's password. The diagram above shows how the login system operates using a Sequence Diagram.

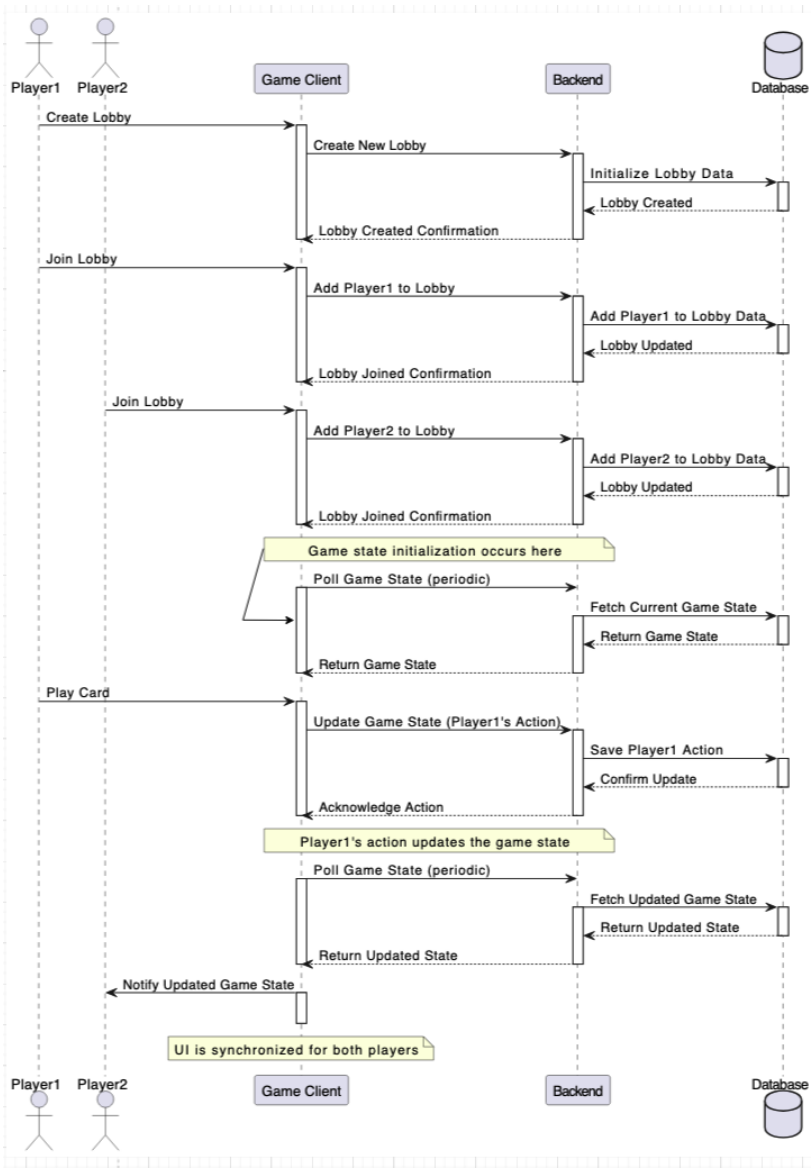

Figure 5: Lobby Sequence Diagram

The above diagram shows the interaction within the system for creating a lobby. From the main menu, the user is able to create their own lobby and invite other players into it using a unique code. This code is linked to the host's lobby and can be shared with up to three other people, allowing them to join the game.

6 Construction

With the construction of the project, we needed several components to work together to ensure the game logic functioned correctly. Since the requirements of the project were already provided by our customer, our focus was on building and integrating the necessary elements into the project.

6.1 Development

6.1.1 Game Logic Sequence

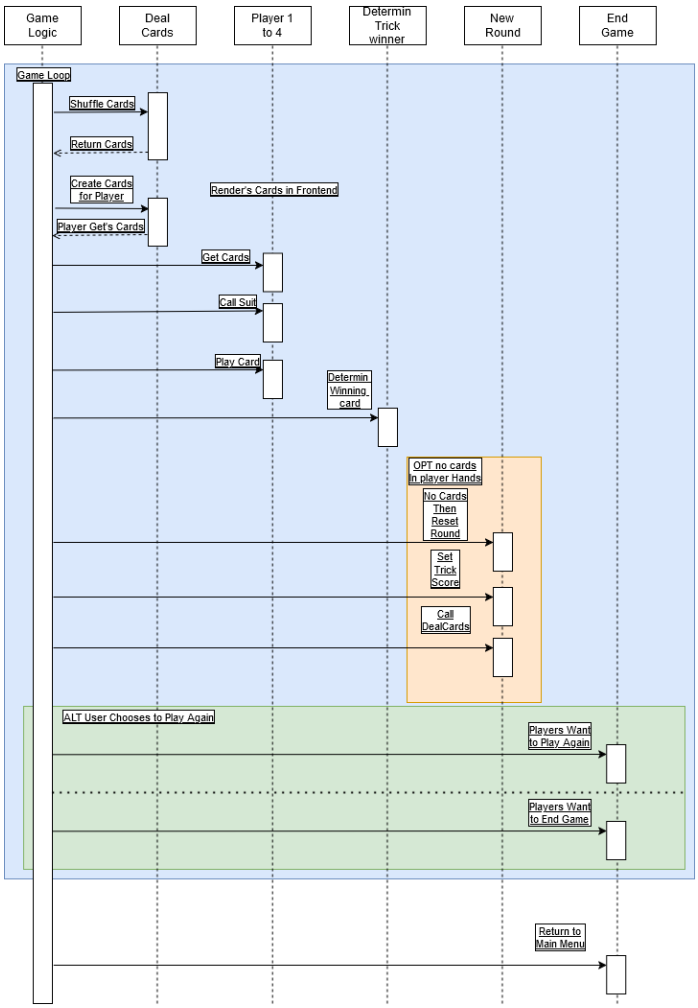

Figure 6: Sequence Diagram Game Logic

The overall logic of the game logic, when playing Sjavs in our application. The Sequence diagram shows the game loop itself in an overview of the game, next section will show the Activities diagram, and in a closer details.

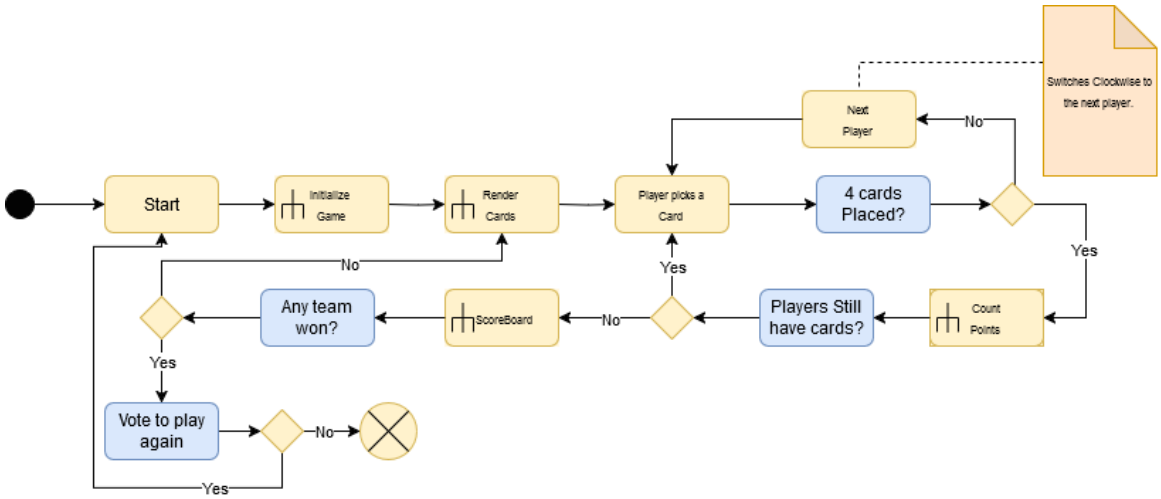

Figure 7: Game Logic Activity Diagram

This section outlines the game logic and the processes that occur within the Sjávs game. Several components work together to ensure the game functions seamlessly, and we will briefly explain these processes. The complete Game Logic Activity Diagram can be found in the **Appendix**.

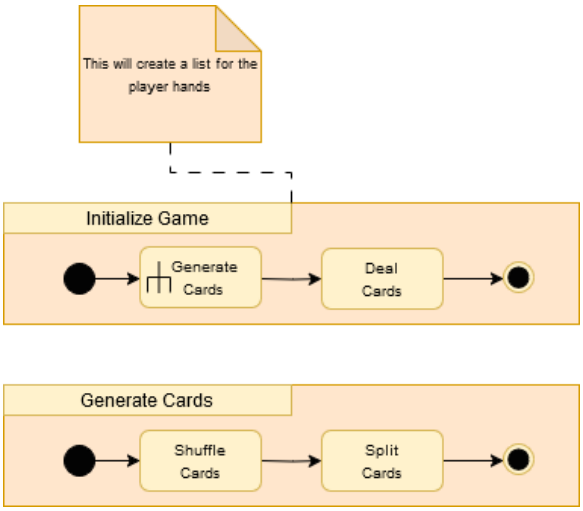

Figure 8: Generate Cards

When a new game is initialized, the system first generates a complete deck of cards and stores it in a single array. This array is then shuffled and divided into four separate arrays, where each array represents a player's hand. These arrays manage the cards assigned to each player. Finally, the cards are rendered on the frontend of the game interface, allowing players to see their respective hands.

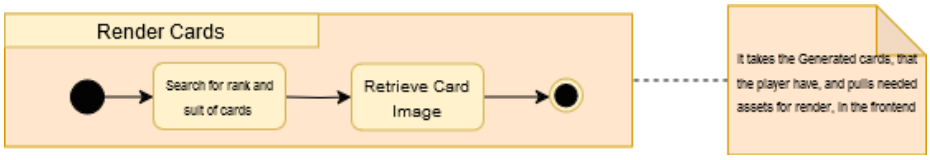

Figure 9: Render Cards

Once the cards have been generated and distributed, they are rendered and added to each player's hand. Each player's hand is represented as an array that stores the assigned cards. On the frontend, the cards are dynamically displayed in a structured layout, allowing players to interact with them during gameplay.

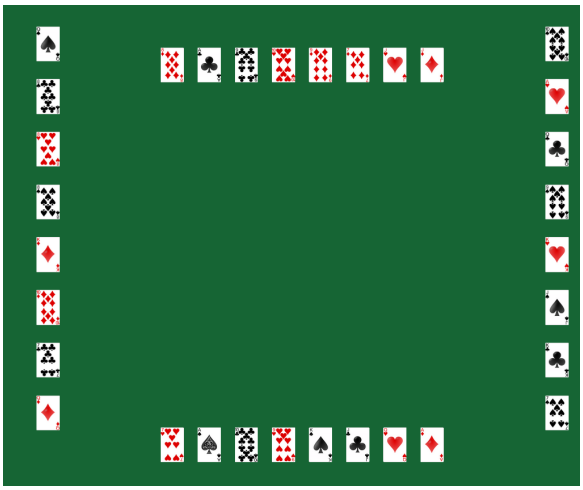

Figure 10: Front page of the game, with the cards rendered

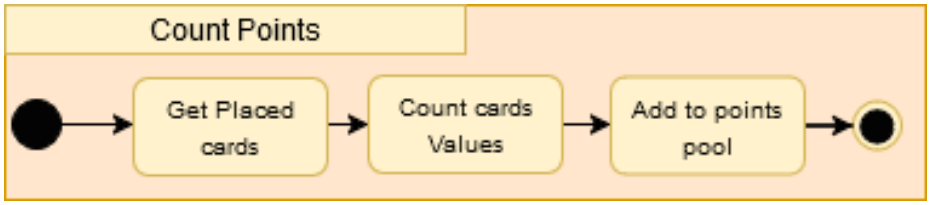

Figure 11: ScoreBoard

Each time all four players have placed a card, the game calculates the total points for the round and assigns them to the winning team. This process repeats until all cards have been played. At this point, the game checks which team has the highest total points and updates the score accordingly.

The figure below shows Team 2 taking the lead after winning the first round.

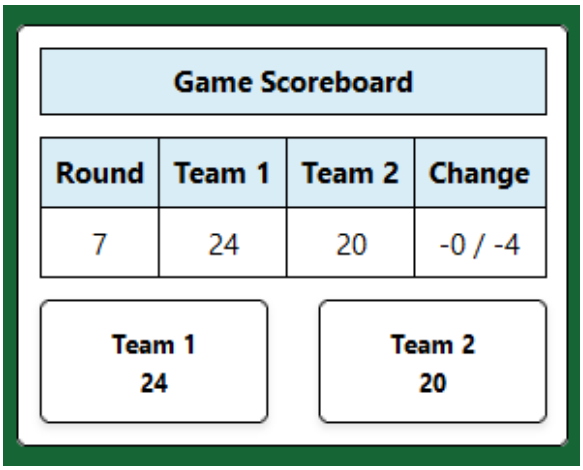

Figure 12: Team 2 winning

This process continues until one of the two teams wins the game. At the end of the game, players are given the option to play another game or reset the game to start fresh.

6.2 Testing and debugging

Testing for the project began with small-scale efforts using **Playwright**. Our initial intention was to maintain a comprehensive test file to ensure the system's functionality. However, due to time constraints, this plan fell behind in the development schedule. By the end of the project, there was insufficient time to implement effective testing for all features.

If we were to start a new project, we would prioritize implementing tests as soon as new methods or features are added to the system. This approach could have saved significant time by identifying and resolving issues earlier in the development process.

The only test that was successfully implemented focused on logging in with a test user. This test navigated to the locally hosted website, arrived at the login screen, filled in the necessary credentials (email and password), and logged in. After logging in, the test checked if the player's hand was visible and if the player had the correct number of cards at the start of the game.

This test was functional and could be easily expanded to run through an entire round of the Sjavs game. For example, it could randomly select cards to play until the scoreboard updated to reflect the round's outcome.

In the future, an end-to-end (E2E) test could follow a similar process: logging in, starting a game of Sjavs, and playing until one team wins. Such a test would validate the game's functionality from start to finish, ensuring a seamless user experience.

6.3 Integration and verification

With the Next.js framework, we were able to set up the project efficiently. Next.js allowed us to separate the frontend into distinct folders using its file-based routing system. Each folder contained a TSX file named page, responsible for loading the necessary assets and components for its specific purpose.

6.3.1 Sjavs Integration

We began by creating the JavaScript (JS) file, which implemented all the game rules and logic. This file served as the core foundation for the game's functionality. Once the core logic was in place, we moved on to developing the frontend of the application.

To manage the game display and components efficiently, we created a dedicated /game folder. This folder was responsible for displaying the game interface to the user, and it included all necessary components within a /components subfolder. As the project progressed, we realized that managing the game logic files could be improved. To address this, we created a new folder within /components called gameFolder. This reorganization ensured that all files controlling the game logic were easy to locate and maintain.

The game page was designed to call the main game component file, which integrated all game logic and displayed the game interface within the application. This setup allowed us to play and test the game directly in the application environment, ensuring that the integration between the game logic and frontend display worked seamlessly.

6.3.2 Clerk

To create a secure environment for our users, we integrated Clerk for user authentication. Clerk simplifies user management and ensures that only authenticated users can access specific parts of the application. Below, we describe the implementation process and how Clerk was utilized throughout our system. Installation and Configuration

We began by installing Clerk into our project using the following command in the Visual Studio Code (VSC) terminal:

```
npm install @clerk/clerk-sdk-node @clerk/nextjs
```

After installation, we configured Clerk by wrapping our application with the Clerk Provider component. This step ensured that Clerk's context and functionality were available across the entire application, allowing us to access user authentication states and manage user sessions seamlessly. Securing Routes and Components

To protect routes and components that required user authentication, we utilized Clerk's higher-order components (HOCs) and hooks. This allowed us to enforce access control efficiently.

Protecting Routes: Routes requiring authentication were secured using Clerk's HOCs. For example, pages containing sensitive user data or restricted functionality were only accessible to signed-in users. Conditional Rendering: With Clerk's hooks like useUser, we dynamically checked the user's authentication status and conditionally rendered content based on whether the user was signed in or not. This ensured that unauthorized users could not view protected content.

Session Management and User Information

Clerk provided robust session management to maintain the user's authentication state across different pages and components:

Session Handling: Once authenticated, Clerk managed the user's session tokens, refreshed sessions automatically, and ensured a smooth navigation experience for signed-in users. Access to User Information: Using Clerk's hooks, such as useUser, we retrieved user-specific information like name, email, and other profile details. This data was used to personalize the user experience, display user profiles, and greet users dynamically.

API Protection with Middleware

To secure backend API routes, we implemented Clerk's middleware. This middleware ensured that only authenticated users could make requests to specific API endpoints.

Clerk middleware in sudo code:

```
IMPORT clerkMiddleware FROM '@clerk/nextjs/server'

DEFINE FUNCTION middleware(auth, request)
  IF request IS NOT a public route THEN
    IF auth.userId IS NOT PRESENT THEN
      RETURN Response with status 401 (Unauthorized)
  END IF
```

```
  RETURN Response with status 200 (OK)
END FUNCTION

EXPORT default middleware
```

The middleware validates the user's authentication session before processing API requests. If the session is valid, the request proceeds; otherwise, access is denied. This approach helped secure sensitive endpoints and ensured that unauthorized users could not interact with protected backend resources. Environment Configuration

To integrate Clerk securely into our environment, we configured the necessary API keys and settings in the .env.local file. This ensured that Clerk could interact with our application while keeping sensitive information secure and isolated.

Summary By integrating Clerk, we ensure a secure and efficient user authentication process. Clerk simplifies user management and enhances security, while Supabase handles the storage and retrieval of user data. This combination provides a robust solution for managing user information and authentication in our application.

7 Deployment

Because we used **Next.js** for our project, we planned to use **Vercel** as our deployment platform. Vercel integrates seamlessly with Next.js and provides an efficient way to host projects on the internet.

7.1 Deployment Strategy

To prepare our project for deployment, we needed to ensure that it could build and optimize successfully for a production environment. We tested this locally by running the following command in the **Visual Studio Code (VSC)** terminal:

```
npm run build
```

This command compiles and optimizes the application for production. Initially, we encountered several issues that prevented the build process from completing successfully. This was due to **poor initial planning** and missing optimizations, which required us to **refactor** parts of the code to meet production requirements.

Once we resolved the local build issues, we attempted to deploy the project using **Vercel**. Our deployment plan was as follows:

1. Connect our **GitHub** repository to Vercel.
2. Allow Vercel to detect the Next.js framework and automatically handle the build and deployment process.
3. Generate a live link to host our application.

Unfortunately, the deployment process did not succeed, as shown in Figure 13:

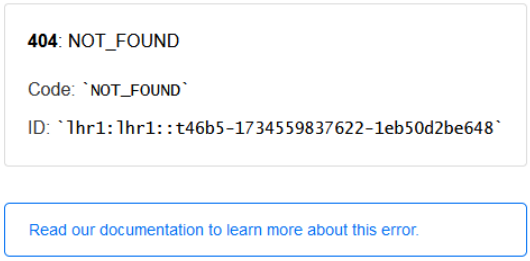

Figure 13: Vercel Deployment Error

7.2 Challenges and Troubleshooting

We attempted several approaches to identify the cause of the deployment issue:

- Analyzed the **Vercel build logs** and deployment summary for specific errors.
- Reviewed the **Vercel documentation** to verify our setup and configurations.
- Ensured that Vercel was deploying from the correct project location.

Despite our efforts, we were unable to pinpoint the root cause of the problem. After carefully considering the time spent troubleshooting, we decided to move on. We recognized that further time spent on this issue would detract from focusing on completing the core features of the main project.

In hindsight, we realize that the deployment process should have been integrated into our workflow from the very beginning. If we were to repeat this project, we would:

1. **Deploy Early and Frequently:** Deploying the project immediately after starting development would allow us to catch deployment issues early. Regular deployments would ensure that we test each change pushed to the repository and resolve errors incrementally.
2. **Automate Testing and Builds:** Implementing automated tests and production builds locally before pushing changes would ensure the codebase remains deployable and free from errors.
3. **Continuous Integration/Continuous Deployment (CI/CD):** Continuous Integration/Continuous Deployment (CI/CD) automates the process of updating deployments and live sites with each push to the Git repository. This pipeline includes automated testing to ensure that only stable and functional code is deployed. If the deployment or tests fail, the push is rejected, preventing broken builds from reaching production. This approach ensures reliable and stable updates while minimizing downtime caused by faulty code. By integrating CI/CD, developers can maintain a consistent deployment process and quickly address any issues introduced during development.

Additionally, deploying the project from the very beginning will allow us to identify any errors caused by new changes (e.g., a push to the repository). By monitoring the deployment logs regularly, we can pinpoint the root cause of issues and resolve them promptly.

7.3 Release management

We initially started with **Bitbucket** but later moved our repository to **GitHub** when we encountered difficulties deploying the project with **Vercel**.

To better manage the project, an efficient use of branches and the early integration of **Continuous Integration/Continuous Deployment (CI/CD)** pipelines would have been ideal. This approach would have ensured that only high-quality, functional code was added to the repository. CI/CD would have prevented code that broke the application from being pushed to the main branch, maintaining a stable and efficient development process.

7.4 Describe the final product

The final product was not approved by the customer due to the absence of critical functionalities required for the project.

While we successfully implemented several features, the product remains incomplete. We have a working game that can be played on a single computer, and we can retrieve and display information from the DBMS, such as the number of wins and losses for each player. Additionally, we implemented a lobby system where players can create a lobby, share a code with others, and view who has joined the lobby.

However, several essential features are still missing. The most significant shortcoming is the lack of a working multiplayer system, which was the customer's primary request. Although players can connect to the same game lobby, the necessary logic to start the game and play together on different computers has not been implemented. Furthermore, the absence of multiplayer functionality means that the tournament feature, another key customer requirement, is also not available.

8 Feedback

8.1 User Testing and Feedback

We only received feedback from our customer, as we were unable to implement a working multiplayer feature for the project. Had we achieved this, we would have requested additional users to test the project and provide suggestions for improvement.

Pál, our customer, expressed dissatisfaction with the appearance of the cards. Specifically, the Ace, King, Queen, and Jack were difficult to distinguish due to the card models we had chosen.

If we had involved other users for testing, we would have provided them with a checklist to guide their feedback. An example checklist is as follows:

1. **Menu Layouts:** What is your opinion on the layout? What could be improved upon?
2. **Navigation:** Is navigating through the menus confusing?
3. **Sjávs Game:** Is all the necessary information displayed clearly, or is there something that needs to be highlighted better?
4. **General Opinion:** Is there anything else you think could be improved?

Using a checklist like this, users would be able to provide structured feedback to help us enhance the overall user experience.

8.1.1 Customer Feedback

For this project, we only received feedback from the customer, Pál.

Pál approved of the game's functionality, particularly how it enforced the rules of Sjávs during gameplay. He also appreciated the layout of the website, noting that it was easy to navigate and understand where the user was located. However, he pointed out several areas for improvement:

1. **Interface and Visuals:** Pál found it challenging to track the flow of the game, particularly after a round ended, as it was unclear whose turn it was to place a card.
2. **Game Feedback:** When a player attempted to place an invalid card, there was no prompt or message explaining what they were doing wrong.

8.2 Missing Features and Future Implementation Plan

The following are the missing features from the project, along with a detailed plan for how they could have been implemented, the methods of implementation, and the steps we would take to test them.

8.2.1 Tournament Functionality

- **Plan:** Add a tournament mode where users can create or join tournaments. Hosts can set rules and fees, and the system will handle prize pool distribution.
- **Implementation:** Expand the current lobby system to include tournament management features. Add database entries for participants, results, and prize allocation.
- **Testing:** Simulate tournaments to ensure smooth participant progression and correct prize distribution.

8.2.2 Multiplayer Capabilities

- **Plan:** Enable real-time gameplay across multiple devices using WebSockets or similar technologies.
- **Implementation:** Set up a WebSocket server to synchronize game states across players. Update the game logic for real-time updates.
- **Testing:** Conduct stress tests with multiple players to ensure smooth and stable communication.

8.2.3 Post Stats on Social Media

- **Plan:** Allow users to share their game stats on platforms like Twitter or Facebook.
- **Implementation:** Add a "Share Stats" button on the leaderboard or player profile. Use social media APIs to generate pre-formatted posts with stats.
- **Testing:** Verify that API calls work properly and posts are displayed correctly on different platforms.

8.2.4 Game Tutorial

- **Plan:** Create an interactive tutorial to guide players through Sjávs rules and gameplay mechanics.
- **Implementation:** Develop a tutorial mode with step-by-step instructions, animations, and tooltips highlighting key elements.
- **Testing:** Gather feedback from new users to ensure the tutorial is easy to understand and comprehensive.

8.2.5 Improved Interface

- **Plan:** Enhance the interface to make it easier for players to track moves and game progress.
- **Implementation:** Add visual indicators (e.g., player highlights or turn markers) and animations to clarify actions and scoring.
- **Testing:** Conduct usability tests to confirm that the updated interface improves clarity and ease of use.

9 Review

This section reflects on my own experiences within the project, how it was handled, areas I wish we had focused more on, and what I could have done to improve the final outcome.

9.1 Describe the Overall Team Experience

Working together with Hanus and Magnus has been a positive experience. While the final product did not meet the customer's requirements, I believe we collaborated effectively and gained valuable insights throughout the process.

9.1.1 What Went Wrong

I underestimated the complexity of the project, particularly the challenges posed by working with unfamiliar tools and concepts. Key issues included:

- **Framework Challenges:** As a team, we had no prior experience with **Next.js**, which required significant additional time to research and learn during the development process.
- **Lack of Familiarity with Software Engineering Principles:** Applying theoretical concepts such as Agile methods, workboards, and SOLID principles proved difficult, as this was our first time utilizing them in a practical context.
- **Programming Language Barriers:** Learning **JavaScript** and **TypeScript** simultaneously with project development created setbacks.

9.1.2 What Worked Well

Despite the challenges, the team accomplished key foundational tasks that advanced the project:

- **Clerk Integration:** Hanus successfully implemented the **Clerk** system, enabling efficient user authentication and management.
- **Backend Connectivity:** Magnus made significant progress in connecting the application to the database, which supported player statistics and data storage.

These contributions helped establish the core structure of the project and provided a foundation for further development.

9.1.3 How Could You Improve

With better planning and prioritization, I believe we could have achieved the customer's expectations. Personally, I could have improved by:

- **Seeking Collaboration:** Instead of spending excessive time solving problems alone through trial and error, I could have collaborated more with my teammates to resolve issues efficiently.
- **Focusing on Assigned Tasks:** I frequently switched between different aspects of the project without completing my current tasks, which created inefficiencies. Establishing clear priorities and focusing on one task at a time would have improved my productivity.
- **Better Planning:** Creating detailed plans and timelines for my assigned tasks would have helped me stay organized and track my progress more effectively.

9.1.4 Final Verdict of the Team

Working with Hanus and Magnus was an enjoyable and rewarding experience. Although we faced challenges, I believe we all demonstrated resilience and adaptability. This project was a valuable learning experience, and we collectively identified areas for improvement that will benefit us in future projects. Despite the difficulties, the team showed determination and collaboration, which I appreciate.

10 Conclusion

10.1 Summary of Key Findings and Recommendations

This project marked my first experience working with frameworks, and while i do not believe that we fully utilize the potential of **Next.js**, I can see its significant benefits. I was also intrigued by the testing system **Playwright**, which I had never used before, as my previous experience was limited to **JUnit**. I highly recommend **Playwright** due to its versatility. Additionally, understanding how **Clerk** works and how it benefits application systems was very insightful.

10.2 Lessons Learned

Working with new software, extensions, and the principles of software engineering has been an eye-opening experience. While it was challenging to work with these tools for the first time, I can see how they, when used correctly, can make any project far more manageable and efficient. I am confident that I will apply these concepts in future projects.

10.3 Future Work

Everything I have learned from this project will be useful in the future. Although I am uncertain if I will use **Next.js** again, this will depend entirely on the requirements of the project I work on. For future projects, I will ensure that I have more than just basic, functional knowledge of the programming language or framework I use. This was one of the most significant setbacks we faced in this project, and I will strive to avoid repeating this mistake.

11 Appendix A

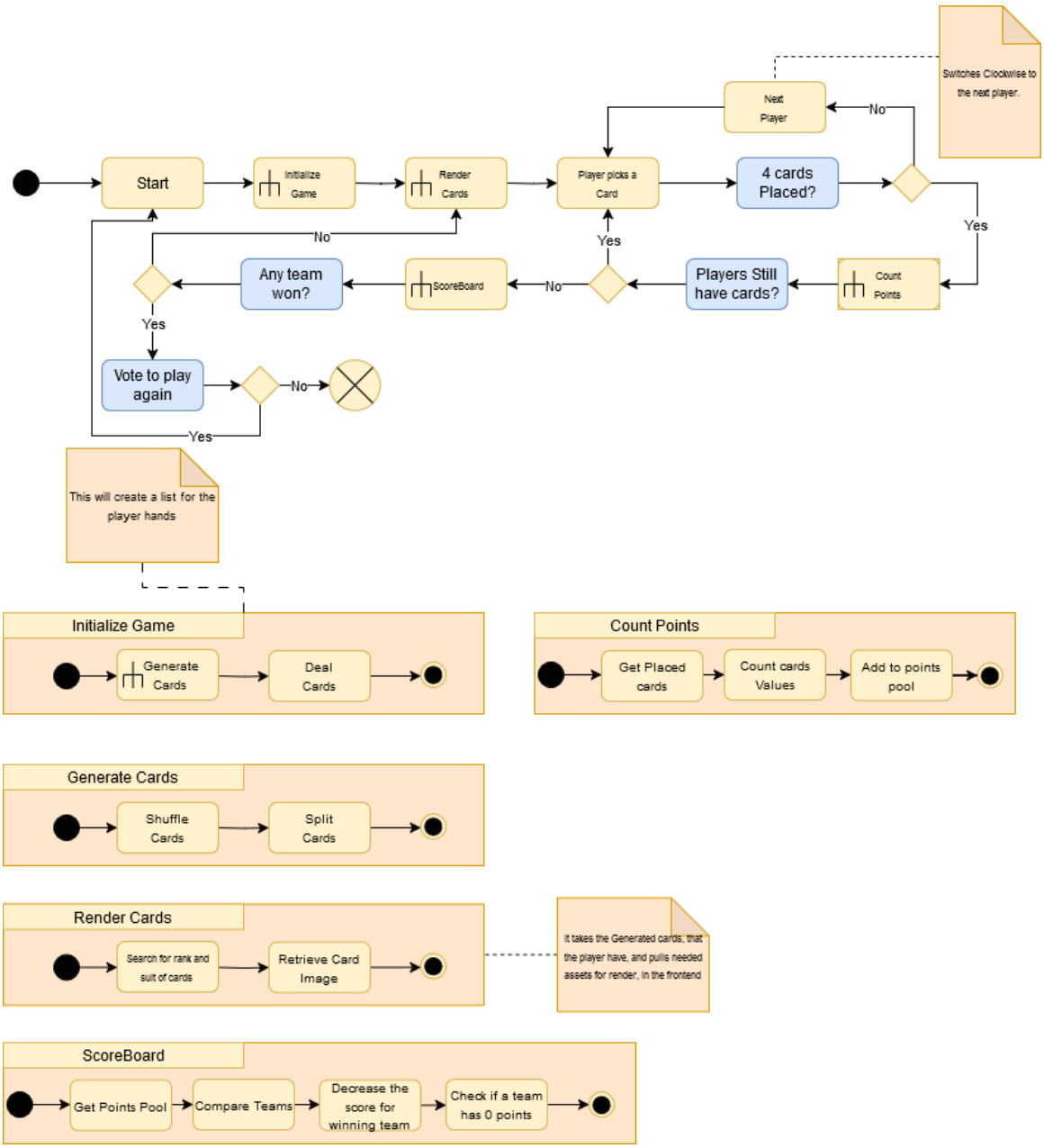

Figure 14: Game Logic Activity Diagram

12 Appendix B

12.1 First Meeting

Summary of Meeting with Pál on November 5

After our meeting with Pál, we gained a clearer understanding of how the game should function. The main points we discussed are as follows:

Player Requirements

- **Minimum Players**: The game should support a minimum of four players.
- **Social Integration**: Players should be able to connect with their Facebook profiles.
- **Interactive Learning**: To make it easier for players to learn the game, there should be interactive messages that provide guidance as they play.
- **Tournament Feature**: Pál expressed interest in offering a paid option for players to join tournaments, which would be created and managed by an admin. While we noted this request, we agreed to focus on developing the core game first, without these additional features.

Key Requirements

- **Trump Suit Selection**: The trump suit should default to clubs, but if it's any other suit, the player should have the option to choose it.
- **Card Selection Rules**: Players should only be able to select valid cards, preventing them from choosing cards that aren't allowed by the game rules.
- **Lobby System**: There should be a system to create and join game lobbies.
- **Game Layout**: The game layout should be designed so that the player's teammate is positioned across from them. The player should be able to view their own cards, with the opposing team's cards displayed on the sides. Additionally, two leaderboards should be visible at all times.

This summary reflects our conversation with Pál, who requested a digital version of the card game "Sjavs." We aim to first develop the basic functionality of the game, with the possibility of adding these extra features later.

12.2 Second Meeting

After our second meeting with Pál on December 12th, he expressed an understanding of the challenges we faced in implementing his application. He accepted that the game would not include multiplayer functionality and would only be playable locally. However, he highlighted several key points of critique that needed to be addressed to create a minimal viable product:

Player Turns Clarity: It should be more apparent on the screen who each player is. Clockwise Turns: Player turns should progress clockwise rather than counterclockwise, as they currently do. First Turn Flexibility: At the start of each round, the player whose turn it is should be able to choose any card to play. Points Scoring Adjustment: Points should not count double after the first round. Score Table: The game should include a table displaying both overall scores and round-by-round scores.

We assured him that these changes would enable us to deliver a minimal viable product and committed to making an effort to implement these corrections.

12.3 Third Meeting

On December 16th, we had our final discussion with Pál regarding the project. While he acknowledged the improvements we implemented since our last conversation, he still had a few critiques and suggestions for further enhancements:

Display Last Trick Score: Currently, the application does not show the score of the last trick, which he highlighted as an important feature.

Card Layouts: He suggested redesigning the card layouts for better visibility, including incorporating cards with images to make them more visually appealing.

Player Visibility: He emphasized the need for a clearer display of player identities to improve the overall clarity of the game interface.

Simplified Trick Score: He noted that showing the score for every trick is unnecessary. Instead, only the final trick score before starting a new round should be displayed.

Card Privacy: To prevent cheating, he requested that players should not be able to see each other's cards.

Turn Indication: He recommended a more graphical representation to clearly indicate which player's turn it is.

General User Experience: He felt that the user experience needed overall improvement to make the game more engaging and intuitive.

The biggest critique, however, was the absence of multiplayer functionality, which he considered essential for the game's success.

Despite these critiques, Pál was understanding of our limitations, recognizing the challenges we faced due to our lack of experience and the constraints of working with a small team.